BLOCKCHAIN
CODING EXAM PREPARATION

Detailed solutions, line-by-line explanations, test cases, and likely practice questions

---

Main topic: Blockchain Integrity Verification — implementing `validate_chain()`

Includes: complete solution, why each check is needed, nonce behavior, demonstration, common bugs, and exam-style questions with answers.

How to study:
1. Understand the main solution.
2. Trace the nonce example carefully.
3. Attempt the practice questions before reading their answers.
4. Finish with the revision sheet.

This guide follows the lab question's requirements rather than assuming a particular ZIP codebase. Code examples use a common dictionary-based block structure; adapt field names and database methods to the starter code supplied in the exam.

1. Understand the lab question

Implement `validate_chain()` inside a `Blockchain` class. Return `True` only if every required check passes; otherwise return `False`.

No.	Required check	Purpose
1	Blockchain is not empty	A chain with no blocks cannot satisfy the genesis-block rule.
2	Genesis height is 1 and previous hash is ZERO_HASH	Confirms the chain starts correctly.
3	Every height and previous hash is correct	Detects misnumbered blocks and broken links.
4	Recalculated hash equals stored hash	Detects header data that does not match the saved digest.
5	Hash satisfies TARGET	Checks the lab's proof-of-work rule.

Illustrative structure

```
Block 1 (genesis): Height=1, previous hash=ZERO_HASH, stored hash=H1
Block 2:           Height=2, previous hash=H1,        stored hash=H2
Block 3:           Height=3, previous hash=H2,        stored hash=H3
```

A block after the genesis block must refer to the hash of the block immediately before it.

Assumptions for the examples: each block is a dictionary with `Height` and `BlockHeader`. The header contains `version`, `prev_block_hash`, `merkle_root`, `timestamp`, `bits`, `nonce`, and `blockHash`. A function `hash256(bytes)` and constants ZERO_HASH and TARGET are available.

2. Complete answer: validate_chain()

Place this method inside the Blockchain class. It assumes the dictionary fields described on the previous page.

```
def validate_chain(self):
    # Step 1: Load all blocks
    chain = self.read_from_disk()

    # Step 2: An empty chain is invalid
    if not chain:
        return False

    # Step 3: Check the genesis block
    first_block = chain[0]
    first_header = first_block["BlockHeader"]

    if first_block["Height"] != 1:
        return False

    if first_header["prev_block_hash"] != ZERO_HASH:
        return False

    # Step 4: Check every block
    for index, block in enumerate(chain):
        header = block["BlockHeader"]

        # Check expected height
        if block["Height"] != index + 1:
            return False

        # Determine expected previous-block hash
        if index == 0:
            expected_prev_hash = ZERO_HASH
        else:
            previous_block = chain[index - 1]
            expected_prev_hash = (
                previous_block["BlockHeader"]["blockHash"]
            )

        # Check the link
        if header["prev_block_hash"] != expected_prev_hash:
            return False

        # Mining increments nonce after hashing.
        # Recover the nonce used to create the saved hash.
        nonce_used = header["nonce"] - 1

        # Recreate the exact hash input
        hash_data = (
            str(header["version"])
            + header["prev_block_hash"]
            + header["merkle_root"]
            + str(header["timestamp"])
            + str(header["bits"])
            + str(nonce_used)
        ).encode("utf-8")

        calculated_hash = hash256(hash_data)

        # Compare calculated and stored hashes
        if calculated_hash != header["blockHash"]:
            return False

        # Check the mining target
        if not calculated_hash.startswith(TARGET):
            return False

    # Every block passed every check
    return True
```

Nonce warning: use `nonce - 1` only when the mining code increments the nonce after calculating the successful hash. If the code increments only after a failed hash, use the saved nonce directly.

3. Detailed explanation

A. Read the chain and reject an empty chain

```
chain = self.read_from_disk()
if not chain:
    return False
```

`read_from_disk()` represents the storage-reading method. An empty list is false-like in Python, so `if not chain` catches it. The early return prevents attempts to access a nonexistent first block.

B. Check the genesis block

```
first_block = chain[0]
first_header = first_block["BlockHeader"]

if first_block["Height"] != 1:
    return False
if first_header["prev_block_hash"] != ZERO_HASH:
    return False
```

Python indexes start at 0, so `chain[0]` is the first block. The lab requires its height to be 1 and its previous hash to be ZERO_HASH.

C. Check block heights

```
for index, block in enumerate(chain):
    if block["Height"] != index + 1:
        return False
```

`enumerate()` provides both index and block. Since list indexes start at zero but heights start at one, use `index + 1`. For indexes 0, 1, 2, expected heights are 1, 2, 3.

D. Check links

```
if index == 0:
    expected_prev_hash = ZERO_HASH
else:
    previous_block = chain[index - 1]
    expected_prev_hash = previous_block["BlockHeader"]["blockHash"]

if header["prev_block_hash"] != expected_prev_hash:
    return False
```

The genesis block points to ZERO_HASH. Every other block points to the stored hash of the preceding block. A mismatch means the chain's links are broken.

E. Recalculate the hash

```
hash_data = (
    str(header["version"])
    + header["prev_block_hash"]
    + header["merkle_root"]
    + str(header["timestamp"])
    + str(header["bits"])
    + str(nonce_used)
).encode("utf-8")
calculated_hash = hash256(hash_data)
```

The field order and conversions must exactly match the mining code. `str()` permits concatenation of numeric values with strings. `encode('utf-8')` converts the result into bytes for the hash function.

F. Compare hash and target

```
if calculated_hash != header["blockHash"]:
    return False
if not calculated_hash.startswith(TARGET):
    return False
return True
```

The first check ensures the saved hash matches the header data. The second checks the lab's target prefix. The final True must be outside the loop, so every block is checked before acceptance.

4. The nonce detail

The lab explicitly says the mining code increments the nonce after calculating the hash. That means the saved nonce can be one greater than the nonce used for the successful hash.

```
# Simplified original pattern
while self.blockHash[0:len(target)] != target:
    self.blockHash = hash256(data_containing(self.nonce))
    self.nonce += 1
```

Event	Nonce
Nonce used for successful hash	3
Increment after hashing	3 + 1
Saved nonce	4
Nonce needed for recalculation	4 - 1 = 3

Alternative: increment only on failure

```
while True:
    block_hash = calculate_hash(self.nonce)

    if block_hash.startswith(target):
        break

    self.nonce += 1
```

In this alternative, the successful hash exits the loop before incrementing. The saved nonce is exactly the nonce used for the hash, so the validator uses the stored nonce directly.

Exam rule: Do not memorize that the nonce must always be decremented. Inspect the mining loop and reproduce its behavior.

The original pattern also assumes the initial blockHash does not already satisfy the target before the first loop iteration. A robust mining loop calculates a hash, checks it, and increments only after failure.

5. Required demonstration

The requested output is:

```
Case 1: Original blockchain
Blockchain Valid: True

Case 2: After modifying a block's nonce
Blockchain Valid: False
```

The following is general test logic. Replace `save_chain()` with the actual database save/update method provided in the exam files.

```
import copy

blockchain = Blockchain()

print("Case 1: Original blockchain")
print("Blockchain Valid:", blockchain.validate_chain())

original_chain = blockchain.read_from_disk()
tampered_chain = copy.deepcopy(original_chain)

tampered_chain[0]["BlockHeader"]["nonce"] += 1

# Replace with the database's actual write/update method
blockchain.save_chain(tampered_chain)

print("\nCase 2: After modifying a block's nonce")
print("Blockchain Valid:", blockchain.validate_chain())

# Restore the original data after testing
blockchain.save_chain(original_chain)
```

`copy.deepcopy()` makes an independent copy of nested lists and dictionaries. This lets the test change the nonce without changing the backup. Restore the original data after the test.

If Case 1 is False: check field order, hash algorithm, nonce behavior, and data structure. **If Case 2 is True:** check that the modified chain was saved and that the validator recalculates the hash.

6. Practice question 1 — Validate heights

Question: Return False if the chain is empty or any block's height is not sequential.

```
def validate_heights(self):
    chain = self.read_from_disk()
    if not chain:
        return False

    for index, block in enumerate(chain):
        expected_height = index + 1
        if block["Height"] != expected_height:
            return False

    return True
```

Why it works: It rejects an empty chain, compares every height with index + 1, returns False on the first mismatch, and returns True only after all blocks pass.

Trace: heights [1, 2, 4] fail at index 2 because the expected height is 3.

7. Practice question 2 — Validate previous-hash links

Question: Check that the genesis block uses ZERO_HASH and every other block references the previous block's stored hash.

```
def validate_links(self):
    chain = self.read_from_disk()
    if not chain:
        return False

    for index, block in enumerate(chain):
        actual = block["BlockHeader"]["prev_block_hash"]

        if index == 0:
            expected = ZERO_HASH
        else:
            expected = (
                chain[index - 1]["BlockHeader"]["blockHash"]
            )

        if actual != expected:
            return False

    return True
```

Why it works: The first block has no predecessor. Later blocks use index - 1 to access the previous block. This checks links only; it does not independently prove that stored hashes are correct.

8. Practice question 3 — Validate one block's hash

Question: Recalculate a block's hash and compare it with its stored hash.

```
def validate_block_hash(block):
    header = block["BlockHeader"]
    nonce_used = header["nonce"] - 1

    data = (
        str(header["version"])
        + header["prev_block_hash"]
        + header["merkle_root"]
        + str(header["timestamp"])
        + str(header["bits"])
        + str(nonce_used)
    ).encode("utf-8")

    calculated_hash = hash256(data)
    return calculated_hash == header["blockHash"]
```

The helper extracts the header, reconstructs the mining input, calculates a fresh digest, and returns the comparison. If the mining code stores the exact successful nonce without incrementing it, use `header['nonce']` directly.

9. Practice question 4 — Check the target

Question: Check whether a hash satisfies a target prefix.

```
def validate_target(calculated_hash, target):
    return calculated_hash.startswith(target)
```

Target	Hash	Result
0000	0000AB12...	True
0000	0001AB12...	False
abc	abcdef123...	True
abc	0abcdef123...	False

This follows the lab's prefix-based target rule. If the problem defines a numeric target, use a numeric comparison instead.

10. Practice question 5 — Find the return bug

Question: Why is the following code wrong?

```
def validate_heights(self):
    chain = self.read_from_disk()
    for index, block in enumerate(chain):
        if block["Height"] != index + 1:
            return False
        return True
```

Answer: `return True` is inside the loop, so the function may accept the chain after checking only its first block.

Fix: reject an empty chain and move `return True` after the loop.

```
def validate_heights(self):
    chain = self.read_from_disk()
    if not chain:
        return False

    for index, block in enumerate(chain):
        if block["Height"] != index + 1:
            return False

    return True
```

11. Practice question 6 — Detect nonce tampering

Question: Why should changing a block's nonce make validation fail if its stored hash is unchanged?

```
Original nonce: 12
Stored hash: hash(header data using nonce 12)

Modified nonce: 13
Recalculated hash: hash(header data using nonce 13)

Recalculated hash != stored hash
Result: False
```

The nonce is part of the hash input. Changing it normally changes the digest, so recalculation no longer matches the saved hash. For this lab, account for the increment-after-hashing behavior when determining the nonce to use.

12. Practice question 7 — Why can tampering break later blocks?

Question: If Block 2 changes, explain why Block 3 can become invalid.

```
Before:
Block 2 stored hash = H2
Block 3 previous hash = H2

After changing Block 2:
Block 2 recalculated hash = H2_NEW
Block 3 previous hash = H2
```

Block 2 fails its own hash check if its recalculated hash no longer matches the stored hash. Block 3's link can also fail because it still references H2, not H2_NEW. Hash links help make tampering detectable.

If someone recalculates and re-mines later blocks, additional network consensus rules matter; hash validation alone is not a complete consensus system.

13. Practice question 8 — Why use SHA-256?

Question: Explain why a cryptographic hash helps verify blockchain integrity.

- Deterministic: the same input gives the same digest.
- Fixed length: SHA-256 produces a 256-bit digest, usually displayed as 64 hexadecimal characters.
- Avalanche effect: a small input change usually changes many digest bits.
- Preimage resistance: finding an input from its digest is computationally infeasible in general.
- Collision resistance: finding two different inputs with the same digest is computationally infeasible.

The validator must use the exact hashing algorithm specified by the starter code. Some systems use double SHA-256; others use a single SHA-256 call. Never change the algorithm during validation.

14. Common mistakes

Mistake	Why it fails	Correction
Skip empty-chain check	No genesis block can be validated.	Use if not chain: return False.
Use index as height	Index starts at 0; height starts at 1.	Compare to index + 1.
Use current block's hash as its previous hash	A block points to its predecessor.	Use chain[index - 1] for later blocks.
Trust stored hash only	Saved data could be tampered with.	Recalculate and compare.
Ignore TARGET	A hash may not meet mining difficulty.	Check the calculated hash.
Wrong nonce assumption	Hash is reconstructed with a different nonce.	Match the mining loop.
Different field order	Different input creates a different digest.	Use exact field order and encoding.
Return True inside loop	Later blocks may never be checked.	Return True after the loop.
Fail to restore test data	The test may leave the chain corrupted.	Back up and restore the original.

15. Short theory answers

What is a genesis block?

The first block. In this lab it has height 1 and previous hash ZERO_HASH.

What is a nonce?

A value changed during mining to search for a hash that meets the difficulty requirement.

What is block height?

The sequential position of a block. This lab starts at 1.

What is a previous-block hash?

A hash reference linking a block to its predecessor.

Why return False immediately?

One failed required condition is enough to reject the chain; further checks are unnecessary.

Why return True after the loop?

It ensures every block has been checked before the chain is accepted.

16. Final five-minute revision sheet

Memorize this sequence:

```
1. Read chain.
2. If empty: return False.
3. Genesis height == 1.
4. Genesis previous hash == ZERO_HASH.
5. For every block:
      height == index + 1
      previous hash == ZERO_HASH for genesis
                       or previous block's stored hash
      recalculate hash from exact header data
      calculated hash == stored hash
      calculated hash satisfies TARGET
6. If all checks pass: return True.
```

Python pattern	Meaning
if not chain:	Detect empty or otherwise false-like chain value.
enumerate(chain)	Get both index and block.
index + 1	Convert zero-based index to one-based height.
chain[index - 1]	Access previous block when index > 0.
.encode('utf-8')	Convert a string to bytes.
.startswith(TARGET)	Check a required string prefix.
return False	Reject immediately when a required check fails.
return True after loop	Accept only after every block passes.

Exam strategy: Underline each explicit requirement, turn it into a Boolean check, use the supplied field names, and test edge cases: empty chain, incorrect genesis, wrong height, broken link, hash mismatch, and target mismatch.

The central lesson: a valid blockchain must have the correct structure, correct links, correct recalculated hashes, and hashes satisfying the mining target. Checking only one condition is not enough.